

Legal RAG System

Complete Build Context & Implementation Prompt
US Tax & Legal Hybrid GraphRAG — Master Prompt for Codex / GitHub Copilot

Prepared for Shyam Suman | July 2026

0. How To Use This Document

This document is a **single, self-contained prompt**. Paste it in full into Codex / GitHub Copilot (or any coding agent) at the start of a session working on this repository. It gives the agent everything it needs: the problem statement, the dataset, the exact JSON contracts already in production, a full narrative of every module already written, the engineering conventions established so far, and a granular, ordered spec for every remaining module up to a working demo and an evaluation report.

Instructions to the agent, read first:

1. Treat Sections 1-7 as ground truth about what already exists. Before writing any new code, open the actual files in the repo (`config.py` , `pdf_parser.py` , `cleaner.py` , `chunker.py` , `metadata_extractor.py`) and confirm they match this description. If `chunker.py` differs from the inferred description in Section 6.3, adapt to the real file — do not overwrite it blindly.
2. Do not change already-working modules' output schemas unless explicitly asked. Every downstream module (embeddings, graph, retrieval) is written against the JSON contracts in Section 5. If a schema must change, change it in one place and update every consumer.
3. Follow the conventions in Section 7 (logging style, `Pipeline` class pattern, docstring style, "drop empty fields", "MERGE not CREATE", provenance-first design) for every new module so the codebase stays consistent.
4. Build in the order given in Section 10 (Sprint Plan). This is a tight, time-boxed project — do not gold-plate; get a thin vertical slice working end-to-end first, then improve quality.
5. Everything must run on **free-tier infrastructure only** (Section 2). Do not introduce a paid API, a paid vector DB, or a paid LLM call anywhere in the pipeline.
6. Two things in the original design notes are **out of date** and have been corrected in this prompt — read Section 2.4 (Grok model deprecation) and Section 8.5 (Elasticsearch native hybrid retriever) before implementing generation or retrieval.

1. Problem Statement (as given)

Project overview. Build a high-precision retrieval-augmented system for the **US Tax and Legal** domain. The system indexes ~100 complex documents and answers natural-language legal questions with summarized answers **grounded in exact legal citations** (document name, page number, section).

Milestone 1 — Data Ingestion & Pre-processing. Parse four document types — Acts (statutory law), Court Judgments (case law), POV (commentary/analysis), and Tax Documents (IRS codes and regulations) — while preserving the page index, because page numbers are the backbone of every citation later in the pipeline. (*Status: mostly complete — see Section 6.*)

Milestone 2 — Search Architecture & Indexing. Implement **hybrid search**: vector (semantic) search, keyword (BM25 / ELK) search, and a **combined hybrid** query. As an advanced component, build a **Knowledge Graph (GraphRAG)** that maps relationships between documents (e.g., which judgment cites which Act, which section belongs to which Act). Standardize the exported knowledge using **OKF (Open Knowledge Format)**. (*Status: not started — Sections 8-9 of this prompt.*)

Milestone 3 — Feature Development (UX). A natural-language Q&A interface; summarization of long rulings; and the two "legal must-haves": every answer must cite the exact **document name**, and provide **page-number references** to the exact source material for human verification. (*Status: not started — Section 8.7-8.9.*)

Milestone 4 — Quality Assurance & Evaluation. You will be given a **Golden Set** (CSV/JSON) of {`query`, `ground_truth_answer`, `source_document`} triples. Run the system against it and report **Retrieval Accuracy** (did it find the right document?) and **Faithfulness** (did the LLM summarize it correctly, without hallucinating?). (*Status: not started — Section 8.10.*)

Deliverables. 1. An architecture diagram: PDF → ELK/Vector DB/Graph → LLM. 2. A working demo: a simple UI or API to test a legal query. 3. An evaluation report with metrics against the Golden Set.

2. Technology Stack & Hard Constraints

This is a **free-tier-only** build, done under time pressure. Every choice below was made to avoid a credit card and avoid rate-limit surprises mid-demo. Do not substitute a paid service unless explicitly told to.

Layer	Choice	Why
PDF parsing	PyMuPDF (<code>fitz</code>) + <code>pytesseract</code> OCR fallback	Already built, free, handles scanned pages
Embeddings	BAAI/bge-small-en-v1.5 via <code>sentence-transformers</code>	Runs locally on CPU, free, 384-dim, strong retrieval quality for its size
Hybrid index	Elastic Cloud (free trial) — one index, BM25 + <code>dense_vector</code> kNN, fused with the native <code>rrf</code> retriever	One trial account does both lexical and vector search — no need to run a second vector DB (see 2.5)
Knowledge graph	Neo4j AuraDB Free	Free forever tier, small size cap, good enough for ~100 documents
Re-ranker	<code>cross-encoder/ms-marco-MiniLM-L-6-v2</code> via <code>sentence-transformers</code>	Free, CPU-friendly, standard choice for RAG re-ranking
LLM	Groq API free tier — see 2.4 for exact model	Free, extremely fast inference, generous-enough daily token quota for a demo
Evaluation	RAGAS (<code>ragas</code> pip package) wired to the same Groq model as judge	Free if the judge LLM call goes through Groq instead of OpenAI
API	FastAPI + Uvicorn	Free, standard
UI	Streamlit	Free, fast to build, good enough for a demo
Knowledge export	OKF (Open Knowledge Format) — plain Markdown + YAML frontmatter	No SDK, no license, just files (see 2.6)

2.1 Dataset

100 documents, four `document_type` values used everywhere downstream: `"acts"`, `"judgments"`, `"pov"`, `"tax"`. Documents are stored under `data/raw/<document_type>/*.pdf` (see Section 4). Roughly split as: Acts (statutory PDFs like `PLAW-118publ15.pdf`), Court Judgments, POV commentary pieces, and Tax/IRS documents.

2.2 No paid infra, ever

- Elastic Cloud **trial**, not a paid subscription — trials expire (usually 14 days), so note the start date somewhere in `config.py` or a `.env` file and re-provision if it lapses before the deadline.
- Neo4j **AuraDB Free** — one free instance per account, small storage/RAM cap, and it **auto-pauses after inactivity**. The agent should build a small "wake-up" retry/backoff into `neo4j_client.py` (first query after a pause can time out — retry once after a short delay).
- Groq free tier has **per-model daily/per-minute token and request caps** (see 2.4). Batch LLM calls conservatively; do not fire one Groq call per chunk during evaluation without a rate-limit-aware retry wrapper.

2.3 Embeddings dimensionality

BAAI/bge-small-en-v1.5 outputs **384-dimensional** vectors. Every place a vector field is declared (Elasticsearch mapping, Neo4j vector index if used, FAISS fallback) must use `dims: 384`. BGE models also expect a retrieval **instruction prefix** on the *query* side only: prefix queries with `"Represent this sentence for searching relevant passages: "` before embedding (do **not** prefix the documents/chunks, only the query) — this measurably improves retrieval quality for BGE models and is a common mistake to skip.

2.4 Correction — Groq model to use

The original design notes (Section 8–9 in the earlier chapter drafts) named `llama-3.3-70b-versatile`. **As of a Groq deprecation notice sent June 17, 2026, `llama-3.3-70b-versatile` (and `llama-3.1-8b-instant`) are deprecated** for free/developer-tier usage, in favor of `openai/gpt-oss-120b` (recommended general replacement) or `qwen/qwen3.6-27b`. **Use `openai/gpt-oss-120b` as the default generation model** in `llm_client.py`, and make the model name a config value (`GROQ_MODEL` in `config.py` / `.env`), not a hardcoded string, so it can be swapped in one place. Before the final submission, re-check <https://console.groq.com/docs/models> for the current authoritative model list and free-tier rate limits, since Groq's catalog changes frequently.

2.5 Correction — one hybrid engine, not two

The original design (chapters 5/8) sketched three independent engines: Qdrant (vector) + Elasticsearch (keyword) + Neo4j (graph). **Given the actual free-tier stack (Elastic Cloud trial + Neo4j AuraDB, no Qdrant)**, do not stand up a fourth service. Modern Elasticsearch (8.14+, GA in 8.16, available on Elastic Cloud including trial tier) has a **retriever framework** with a native `rrf` **retriever** that composes a `standard` (BM25) sub-retriever and a `knn` (dense vector) sub-retriever and fuses them server-side with Reciprocal Rank Fusion in a **single query** — no separate vector database and no client-side RRF code needed for the text-vs-vector fusion. This project therefore has **two** retrieval engines, not three: - **Elasticsearch** = lexical + semantic in one hybrid query (via the `rrf` retriever). - **Neo4j** = relationship graph, used to *narrow the candidate set before* the Elasticsearch query runs (a `terms` filter on `document_id` / `chunk_id`), not as a third input to a fusion formula. A hand-written Python RRF function (Section 8.7.3) is still worth

having, for two reasons: (a) as a fallback if the Elastic Cloud trial tier or account restricts the retriever API, and (b) it is exactly the kind of "explain the internals" detail that comes up in interviews — build it, even if the native retriever is what actually runs in the demo.

2.6 Note on OKF (Open Knowledge Format)

OKF is a real, recent spec: Google Cloud published **OKF v0.1** in June 2026 — a directory of Markdown files with YAML frontmatter, one concept per file, cross-linked with normal Markdown links, forming a knowledge graph that both humans and LLMs can read directly. The **only required frontmatter field is `type`**; recommended optional fields are `title`, `description`, `resource`, `tags`, and `timestamp`. There is no SDK and no server — it is just a folder convention. Section 8.3 gives the exact bundle layout to use for this project.

3. Dataset Description

- **Volume:** ~100 PDF documents total across four `document_type` buckets.
- **Types and what they mean downstream:**
- `acts` — statutory law (e.g. Public Laws like `PLAW-118publ15.pdf`). Rich in `SEC.` headings, `TITLE / CHAPTER` markers, and USC citations.
- `judgments` — court case law. Rich in `Party v. Party` case names, court names, judge names.
- `pov` — commentary/analysis pieces. Rich in author bylines, may reference acts/cases but is not itself authoritative law.
- `tax` — IRS codes/regulations. Rich in `Publication NNN`, `Form NNNN`, `Schedule X`, and tax year references.
- Every document is processed **page by page**, and the page number is carried through the *entire* pipeline (page → chunk → embedding → index → citation) because the whole point of the project is that an answer must be traceable to an exact page.

4. Current Repository Structure (as of Milestone 1)

```
legal-rag-system/  
├── config.py                # central paths + constants (see 4.1)  
├── data/  
│   ├── raw/  
│   │   ├── acts/*.pdf  
│   │   ├── judgments/*.pdf  
│   │   ├── pov/*.pdf  
│   │   └── tax/*.pdf  
│   ├── processed/          # PROCESSED_DATA_DIR  
│   │   ├── pages.json      # output of pdf_parser.py  
│   │   ├── clean_pages.json # output of cleaner.py  
│   │   ├── chunks.json     # output of chunker.py (exists, code not shown here)  
│   │   └── chunks_with_metadata.json # output of metadata_extractor.py  
├── ingestion/  
│   ├── pdf_parser.py        # DONE  
│   ├── cleaner.py           # DONE  
│   ├── chunker.py           # EXISTS — schema known from chunks.json, verify against repo  
│   └── metadata_extractor.py # DONE  
└── (nothing else exists yet)
```

4.1 `config.py` (inferred from imports — verify against the real file)

The other modules import these names from `config.py`, so it must define at least:

```
RAW_DATA_DIR          # Path, e.g. Path("data/raw")  
PROCESSED_DATA_DIR    # Path, e.g. Path("processed")  
PAGES_JSON            # PROCESSED_DATA_DIR / "pages.json"  
CLEAN_PAGES_JSON      # PROCESSED_DATA_DIR / "clean_pages.json"  
CHUNKS_JSON           # PROCESSED_DATA_DIR / "chunks.json"  
CHUNKS_WITH_METADATA_JSON # PROCESSED_DATA_DIR / "chunks_with_metadata.json"  
DOCUMENT_TYPES        # e.g. ["acts", "judgments", "pov", "tax"]  
OCR_TEXT_THRESHOLD    # min chars before OCR fallback triggers in pdf_parser.py
```

Action for the agent: open the real `config.py` first. Then *extend* it (don't replace it) with the new constants every later module needs — a full list is in Section 9 ("Final `config.py` additions").

5. JSON / Data Contracts Already In Production

These schemas are **load-bearing** — every module built from here on reads or writes one of these exact shapes. Field names must match exactly.

5.1 `pages.json` — one object per PDF page

```
{
  "document_id": "9c253ec2-ade9-49a2-8173-48424c39714a",
  "document_name": "PLAW-118publ15",
  "document_type": "acts",
  "source_file": "PLAW-118publ15.pdf",
  "relative_path": "D:\\Projects\\legal-rag-system\\data\\raw\\acts\\PLAW-118publ15.pdf",
  "page_number": 16,
  "character_count": 2423,
  "word_count": 366,
  "page_id": "9c253ec2-ade9-49a2-8173-48424c39714a_PAGE_0016",
  "processed_at": "2026-07-12T03:15:15.835431",
  "text": "137 STAT. 86\\nPUBLIC LAW 118-15-SEPT. 30, 2023\\n..."
}
```

Field notes: `document_id` is one UUID per source PDF (stable across all its pages). `page_id` = `f"{document_id}_PAGE_{page_number:04d}"`. `relative_path` is currently a Windows path baked in at parse time — treat it as informational only, never re-open it from a different OS; re-derive real paths from `RAW_DATA_DIR / document_type / source_file` instead.

5.2 `clean_pages.json` — same shape, `text` field cleaned in place

Produced by `cleaner.py`. Identical schema to `pages.json`; only `text` changes (hyphenation fixed, smart quotes normalized, control characters stripped, whitespace collapsed). Every other field is copied through unchanged.

5.3 `chunks.json` — one object per chunk (multi-page aggregation)

```
{
  "chunk_id": "9c253ec2-ade9-49a2-8173-48424c39714a_CHUNK_0002",
  "document_id": "9c253ec2-ade9-49a2-8173-48424c39714a",
  "document_name": "PLAW-118publ15",
  "document_type": "acts",
  "source_file": "PLAW-118publ15.pdf",
  "chunk_index": 2,
  "page_start": 1,
  "page_end": 2,
  "page_ids": [
    "9c253ec2-ade9-49a2-8173-48424c39714a_PAGE_0001",
    "9c253ec2-ade9-49a2-8173-48424c39714a_PAGE_0002"
  ],
  "character_count": 2882,
  "word_count": 400,
  "text": "5860] VerDate Sep 11 2014 ... 137 STAT. 72 PUBLIC LAW 118-15 ...",
  "chunked_at": "2026-07-12T17:31:02.462508"
}
```

Field notes: chunks can **span multiple pages** (`page_start` .. `page_end`, and the exact `page_ids` list) — this is why every citation later must resolve to a **page range**, not a single page. Chunking clearly uses a **sliding window with overlap** (the Ch.8 design notes say an 80-word overlap) — **confirm the exact target chunk size and overlap by reading `chunker.py`** before writing the embedding/context-budgeting code, since token budgeting (Section 8.8) depends on the real chunk size in words/tokens.

5.4 `chunks_with_metadata.json` — chunk + a `metadata` object bolted on

Same shape as `chunks.json`, plus one new key, `"metadata"`, whose contents vary by `document_type` but always follow the **unified schema** below (only non-empty keys are kept — no `null` / `[]` clutter):

```

"metadata": {
  "usc_citations": ["21 U.S.C. 379j-11"],
  "cfr_citations": [],
  "dates": ["Sept. 30, 2023"],
  "keywords": ["animal", "drug", "amendments", "fees", "manufacturing"],
  "entities": {
    "acts": ["Animal Drug User Fee Amendments of 2023", "Federal Food, Drug, and Cosmetic Act"],
    "code_references": [],
    "cases": [],
    "courts": [],
    "judges": [],
    "author": null
  },
  "sections": ["2301", "2302", "379j-11"],
  "articles": [],
  "chapters": [],
  "titles": ["III"],
  "irs_publications": [],
  "tax_forms": [],
  "tax_years": []
}

```

Not every field appears on every chunk: acts chunks get `sections/articles/chapters/titles`; judgment chunks get `entities.cases/courts/judges`; tax chunks get `irs_publications/tax_forms/tax_years`; pov chunks get `entities.author`. `keywords` is corpus-level **TF-IDF weighted** (common legal boilerplate like "federal"/"secretary"/"united" is down-weighted), computed once across the whole corpus in `build_corpus_idf_scores()`, not per-chunk.

5.5 New schemas this prompt introduces (contracts the agent must build to)

`relationships.json` (Section 8.1 output):

```

[
  {"source": "chunk_9c25...CHUNK_0002", "source_type": "Chunk", "target": "Section_2302", "target_type": "Section",
   "relationship": "MENTIONS"},
  {"source": "Section_2302", "source_type": "Section", "target": "Act_animal-drug-user-fee-amendments-of-2023", "target_type":
   "Act", "relationship": "PART_OF"},
  {"source": "Section_2302", "source_type": "Section", "target": "USC_21-379j-11", "target_type": "USC", "relationship":
   "REFERENCES"}
]

```

`golden_set.csv` (given for Milestone 4 — schema to *expect*, confirm exact columns once provided):

```

question,ground_truth_answer,source_document,expected_pages
"What does Section 2302 amend?","Section 2302 amends the Federal Food, Drug, and Cosmetic Act ...","PLAW-118publ15","16-17"

```

API response schema (Section 8.9):

```

{
  "question": "Explain Section 2302.",
  "answer": "Section 2302 amends ... [SOURCE_1][SOURCE_2]",
  "confidence": 0.87,
  "sources": [
    {"document": "PLAW-118publ15", "pages": [16, 17], "chunk_ids": ["9c25...CHUNK_0002"]}
  ]
}

```

6. Modules Already Built — Detailed Walkthrough

Read this section to understand *why* the existing code looks the way it does, so new modules extend the same philosophy instead of contradicting it.

6.1 `pdf_parser.py` — `PDFParser` class

Responsibility: turn every PDF under `data/raw/<type>/` into page-level text while preserving page numbers, one page = one JSON object.

- Iterates `DOCUMENT_TYPES`, then every `.pdf` under `RAW_DATA_DIR/<type>/`.
- Opens each PDF with `fitz.open(pdf_path)`; assigns **one** `document_id` (**UUID4**) **per PDF**, shared by every page from that PDF.

- Per page: `extract_page_text()` tries native text extraction (`page.get_text("text")`) first; if the page text is empty/whitespace, it **falls back to OCR** (`perform_ocr`) — rasterizes the page at 300 DPI with `page.get_pixmap(dpi=300)`, converts to a PIL `Image`, and runs `pytesseract.image_to_string(image, lang="eng")`. OCR pages are counted separately (`self.ocr_pages`) for the summary log.
- Builds a metadata dict per page (`create_page_metadata`) with the exact fields in Section 5.1, including a deterministic `page_id`.
- Tracks running stats: `total_documents`, `total_pages`, `ocr_pages`, `failed_documents` — a failed PDF (corrupt file, etc.) is logged and skipped, not fatal to the whole run.
- Writes the full list to `pages.json` with `json.dump(..., indent=4, ensure_ascii=False)` (this `ensure_ascii=False` convention matters for legal text with special characters like \$, —, smart quotes — **keep it in every later JSON writer too**).
- Has a `print_summary()` that logs a boxed `"=*60` summary block — **this logging style is used everywhere in this codebase; keep using it** in new pipelines (see Section 7.2).

6.2 `cleaner.py` — `LegalTextCleaner` + `CleanerPipeline`

Responsibility: normalize `pages.json` text without destroying legal citation formatting.

- `fix_hyphenation`: joins words split across a line break (`busi-\nness` → `business`) with a regex on `(\w)-\s*\n\s*(\w)`.
- `normalize_quotes`: converts curly quotes (" ' ' ') to straight ASCII quotes.
- `remove_control_characters`: strips non-printable control chars (`\x00-\x08`, `\x0B`, `\x0C`, `\x0E-\x1F`) but **deliberately does not touch** `\t` and `\n` used for structure.
- `normalize_whitespace`: collapses runs of spaces/tabs, collapses 3+ newlines to 2, strips trailing whitespace before newlines.
- **Deliberately does NOT** lowercase, stem, remove stopwords, or strip punctuation — this is a *legal* corpus; citation-bearing punctuation (`$`, `U.S.C.`, `(a)(1)(A)`) must survive untouched for the regex-based metadata extractor downstream to work.
- `CleanerPipeline` mirrors the `PDFParser` shape: `load_pages` → `clean_pages` → `save_pages` → `print_summary`, and tracks `characters_before` / `characters_after` for the summary.
- Defensive: `page.get("text", "")` and a type check (`if not isinstance(original_text, str)`) guard against malformed input rather than crashing the whole batch.

6.3 `chunker.py` — inferred responsibility (verify against real file)

Not shown in this prompt, but its output contract (`chunks.json`, Section 5.3) tells us it must: - Group consecutive cleaned pages into chunks of a target size (likely ~300–500 words, given the ~400-word chunk in the sample), **allowed to span multiple pages**. - Use a **sliding window with overlap** between consecutive chunks (design notes mention 80 words) so that a fact split across a chunk boundary is not lost — this is *why* `context_builder.py` (Section 8.7.4) must later de-duplicate near-identical overlapping chunks before they reach the LLM. - Preserve `page_start`, `page_end`, and the *full* `page_ids` list per chunk, and generate `chunk_id = f"{document_id}_CHUNK_{chunk_index:04d}"`. - **Before writing any new code that depends on chunk size/overlap (embeddings batch sizing, token budget math), open `chunker.py` and read the real numbers.**

6.4 `metadata_extractor.py` — `MetadataExtractor` + `MetadataPipeline`

Responsibility: enrich every chunk with structured, queryable, **regex-only** metadata (no LLM calls) — this keeps Milestone 1 fully free and deterministic.

Key design ideas worth preserving in every later module: - **Unified schema, sparse population.** Every chunk gets metadata built from the same superset of keys, but `drop_empty_fields()` removes any key that is `None`, `""`, or `[]` before writing — so an Act chunk never carries a stray `"judges": []`. Recurses one level into the nested `"entities"` dict and drops `"entities"` entirely if nothing survives inside it. - **Common vs type-specific split.** `extract_common_metadata()` (USC/CFR citations, dates, act names, code references, keywords) runs on every chunk regardless of type; a dispatcher (`type_extractors` dict keyed by `document_type`) then adds `acts` → sections/articles/ chapters/titles, `judgments` → cases/courts/judges, `tax` → publications/forms/years, `pov` → author. `_merge_metadata()` deep-merges the `"entities"` sub-dict rather than overwriting it, so common entities (`acts`, `code_references`) survive alongside type-specific ones (`cases`, `courts`). - **Corpus-level TF-IDF for keywords**, computed *once* in `build_corpus_idf_scores()` over all chunks (skipped for corpora smaller than 2 chunks), so boilerplate legal language common to nearly every chunk ("federal", "united", "secretary" — all in `STOPWORDS` too, belt-and-braces) is down-weighted and terms distinctive to *this* chunk float to the top. `idf = ln((N+1)/(df+1)) + 1`, standard smoothed IDF. Score = `term_count * idf`, top 8 kept. - **Regex citation extraction is careful about noise.** `extract_act_names` filters out generic non-named matches like `"This Act" / "Congress ... Act"` / anything with fewer than two capitalized words before "Act" (`_is_generic_act_reference`). `extract_case_names` strips leading connector words a greedy regex can swallow (`"In Welch v. Helvering"` → `"Welch v. Helvering"`). `extract_court_names` matches a **fixed reference list**, longest name first, and tracks claimed character spans so `"Supreme Court"` doesn't also fire inside an already-matched `"United States Supreme Court"`. - **Keyword tokenization deliberately excludes digits** (`[A-Za-z]{4,}`) so citation numbers never leak into the keyword list, and known citation/date substrings are stripped from the text *before* tokenizing for keywords, so a citation's own words don't dominate. - **This module is the single source of truth for "what entities exist in this text."** The relationship extractor (Section 8.1) and the query-time entity detector (Section 8.7.2) **must reuse these same regex functions** (import them, don't reimplement them) so that entity names extracted from chunks and entity names detected in a user's question are guaranteed to match. - `MetadataPipeline` again

mirrors `load → build_idf → enrich → save → print_summary`, and adds a `field_coverage: Counter` that logs what % of chunks got each field populated — a good sanity check to replicate for the relationship extractor and the graph loader (log a node/edge-type coverage table the same way).

6.5 Design principles this repo already follows — keep them going

1. **Regex/deterministic before LLM.** Milestone 1 explicitly avoided any LLM/NLP model calls in metadata extraction ("Avoid LLM inference initially... Add LLM-generated semantic edges only in later versions"). Relationship extraction (Section 8.1) follows the same rule: it is derived from already-verified metadata, not from raw text or an LLM call.
2. **Class + Pipeline pattern.** An `XExtractor / XCleaner / XParser` class holds pure, independently-testable methods; an `XPipeline` class orchestrates `load → process → save → print_summary`. Keep this shape for every new stage (`RelationshipExtractor / RelationshipPipeline`, `EmbeddingGenerator / EmbeddingPipeline`, etc.).
3. **Provenance always survives.** Every artifact downstream carries enough IDs to walk back to `document_id → document_name → page_ids → exact page number`. Never build a schema that drops this.
4. **Idempotent, incremental-friendly outputs.** JSON files are fully regenerated each run today, but design the graph/index loaders (Section 8.2, 8.4) to use `MERGE`-style upserts keyed on a stable ID, so re-running ingestion after adding new documents does not require a full rebuild.
5. `json.dump(..., indent=4, ensure_ascii=False)` **everywhere**, and the same `"="*60` boxed logging summary at the end of every pipeline stage.

7. Cross-Cutting Conventions (apply to every new module)

- **Logging:** `logging.basicConfig(level=logging.INFO, format="%(asctime)s | %(levelname)s | %(message)s")`, one module-level `logger = logging.getLogger(__name__)`. End every pipeline run with a boxed `print_summary()`.
- **Docstrings:** short triple-quoted purpose statement per method, written the same terse style as the existing code (see `metadata_extractor.py`).
- **Config, not hardcoding:** every path, model name, API key env var, index name, and Cypher label lives in `config.py` or `.env` — never inline in a module.
- **Secrets:** `GROQ_API_KEY`, `ELASTIC_CLOUD_ID / ELASTIC_API_KEY`, `NEO4J_URI / NEO4J_USER / NEO4J_PASSWORD` all go in a `.env` file loaded with `python-dotenv`; **never commit** `.env`; commit a `.env.example` with blank values.
- **Entity canonicalization:** the same real-world entity can appear in slightly different textual forms (`"21 USC 379j-11"` vs `"21 U.S.C. 379j-11"`). Before creating a graph node or an Elasticsearch filter term, run every extracted entity string through one shared `normalize_entity(text: str) -> str` helper (strip periods after "U", "S", "C"; normalize the en-dash `–` to a hyphen `-`; collapse whitespace; uppercase for the ID, keep original casing for display) so `"21 U.S.C. 379j-11"` and `"21 USC 379j-11"` map to the **same node**.
- **Testing:** add a `tests/` folder with at least one smoke test per new pipeline stage that runs on a tiny fixture (2-3 chunks) and asserts the output schema and non-crash — given the time pressure, favor **breadth** (one test per module) over deep coverage.

8. Remaining Work — Detailed Module Specifications

8.1 Relationship Extraction — `graph/relationship_extractor.py`, `graph/graph_schema.py`

Purpose: turn the *metadata already sitting in* `chunks_with_metadata.json` into an edge list, deterministically, with zero LLM calls (consistent with Section 6.5 rule #1).

Input: `chunks_with_metadata.json` . **Output:** `relationships.json` (schema in 5.5).

Node types (`graph_schema.py`, one small dataclass/Enum per type + a canonical-ID function for each): `Document`, `Chunk`, `Act`, `Section`, `USC`, `CFR`, `Judgment`, `Court`, `Judge`, `IRSPublication`, `TaxForm`, `Keyword`, `POV`.

Deterministic edge rules to implement (straight translation of the metadata already present on each chunk — do not invent new extraction logic, just wire existing fields to edges):

Rule	Source	Target	Relationship
1	<code>Document</code>	<code>Chunk</code>	<code>HAS_CHUNK</code>
2	<code>Chunk</code>	<code>each metadata.entities.acts[i]</code>	<code>MENTIONS</code>
3	<code>Chunk</code>	<code>each metadata.usc_citations[i]</code>	<code>MENTIONS</code>
4	<code>Chunk</code>	<code>each metadata.entities.courts[i]</code>	<code>MENTIONS</code>
5	<code>Chunk</code>	<code>each metadata.entities.judges[i]</code>	<code>MENTIONS</code>
6	<code>Chunk</code>	<code>each metadata.entities.acts[i]</code>	<code>MENTIONS</code>
7	<code>Section</code>	<code>the chunk's entities.acts[0]</code>	<code>MENTIONS</code>
8	<code>Section</code>	<code>each usc_citations co-occurring in the same chunk</code>	<code>REFERENCES</code>
9	<code>Judgment</code>	<code>each entities.acts[i] mentioned anywhere in that document's chunks</code>	<code>CITES</code>
10	<code>Court</code>	<code>Judgment (its source document)</code>	<code>DECIDED</code>

(only when a court name and the judgment come from the same

document) | 11 | Judge | Judgment (its source document) | AUTHORED | 12 | (for document_type == "pov") POV (= source document) | each entities.acts[i] | DISCUSSES | 13 | (for document_type == "tax") Document | each irs_publications[i] | REFERENCES |

Implementation notes: - Work **per document first** (group chunks by `document_id`), then per chunk, so document-level edges (rules 9-13) can look across all of that document's chunks. - Every node ID must be **canonical** (Section 7): e.g. `Act` node ID = `"Act_" + slugify(normalize_entity(act_name))`, `Section` node ID = `"Section_" + normalize_entity(section_number)`, `USC` node ID = `"USC_" + normalize_entity(citation)`. - Every edge dict must carry `chunk_id`, `document_id`, and `document_name` as extra properties (not just `source / target / relationship`) so `neo4j_loader.py` can set them as **relationship properties** — this is the provenance trail the whole project depends on for citations (Section 6.5 rule #3). - Deduplicate edges (same `source`, `target`, `relationship` triple) before writing, but keep a `count` or `chunk_ids: [...]` list on the surviving edge so you don't lose the fact that a relationship was observed in multiple chunks.

8.2 Neo4j — `graph/neo4j_client.py`, `graph/neo4j_loader.py`, `graph/cypher_queries.py`

`neo4j_client.py` : a thin wrapper around the official `neo4j` Python driver.

```
from neo4j import GraphDatabase
class Neo4jClient:
    def __init__(self, uri, user, password):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))
    def run(self, query, **params):
        with self.driver.session() as session:
            return list(session.run(query, **params))
    def close(self):
        self.driver.close()
```

Wrap `run()` in a **retry-once-after-delay** (e.g. 5s) around `ServiceUnavailable` /timeout exceptions — AuraDB Free auto-pauses after inactivity and the very first query after a pause needs longer to come back up (Section 2.2).

`neo4j_loader.py` : reads `relationships.json`, and for every unique node, `MERGE (n:Label {id: $id}) SET n.name = $name, ...` (never `CREATE`, so re-runs don't duplicate). For every edge, `MATCH` both endpoints by `id` then `MERGE (a)-[r:REL_TYPE]->(b) ON CREATE SET r.chunk_ids = $chunk_ids ...`. Batch in chunks of a few hundred rows per transaction (`UNWIND $rows AS row ...`) rather than one `MERGE` per HTTP round-trip — AuraDB Free has modest throughput and this matters for 100 documents' worth of edges. Create indexes/constraints once at startup: `CREATE CONSTRAINT IF NOT EXISTS FOR (a:Act) REQUIRE a.id IS UNIQUE` (repeat per label).

`cypher_queries.py` : a small library of reusable, parameterized queries, at minimum: - Given a list of detected `Section` numbers/ `Act` names/ `USC` citations from a user's question, return every `Chunk / Document` within N hops (graph expansion, Section 8.7.3):

```
MATCH (e)-[*1..2]-(c:Chunk)
WHERE e.id IN $entity_ids
RETURN DISTINCT c.chunk_id AS chunk_id, c.document_id AS document_id
```

- "Which judgments cite Section X of Act Y" (the demo's marquee query, matching the problem statement's own example question) — `Act` → `CONTAINS` → `Section` ← `REFERENCES / MENTIONS` ← `Chunk` ← `HAS_CHUNK` ← `Document` where `document_type = "judgments"`.

8.3 OKF Exporter — `graph/okf_exporter.py`

Export the graph as a portable, human-readable **OKF v0.1** bundle (Section 2.6). Directory layout:

```
knowledge/
├─ acts/<slug>.md
├─ judgments/<slug>.md
├─ sections/<slug>.md
├─ usc/<slug>.md
└─ index.md
```

Each file: YAML frontmatter with the **required** `type` field (`act`, `judgment`, `section`, `usc`, ...) plus the recommended optional fields `title`, `description`, `resource` (path back to the source PDF), `tags` (e.g. `document_type`, related act names), `timestamp`. Body: a short Markdown description plus **normal Markdown links** to related concepts (`[Section 2302](../sections/section-2302.md)`) — this is what turns the folder into a graph a coding agent or another LLM can walk directly, independent of Neo4j. This export is **not** used at retrieval time; it is a portability/inspection artifact for the deliverables (nice to show in the architecture diagram and demo walkthrough).

8.4 Embedding Generation — `embeddings/embedding_generator.py`

- Load `BAAI/bge-small-en-v1.5` once via `sentence_transformers.SentenceTransformer` (CPU is fine for ~100 documents' worth of chunks).
- Read `chunks_with_metadata.json`; embed the **document side with no prefix**, batch (e.g. batch size 32-64) for speed; L2-normalize the vectors (`normalize_embeddings=True`) so cosine similarity $\approx$ dot product, which matters for how Elasticsearch scores kNN.
- **Never embed a query with this same code path without the BGE query prefix** (Section 2.3) — put a `mode: Literal["query", "passage"]` flag on the embedding function used by both `embedding_generator.py` (`mode="passage"`) and `vector_search / hybrid_search` at query time (`mode="query"`) so this is impossible to get wrong by accident.
- Output: either bolt a `"vector": [...]` field directly onto each chunk record (simplest — one `chunks_with_embeddings.json`) or store vectors separately keyed by `chunk_id` if the file gets unwieldy. Given ~100 documents, a single combined JSON file is fine and simpler for the ES bulk-indexer to consume directly.

8.5 Elasticsearch Hybrid Index — `search/elastic_indexer.py`, `search/elastic_search.py`

Index mapping (create once, at startup):

```
{
  "mappings": {
    "properties": {
      "chunk_id": {"type": "keyword"},
      "document_id": {"type": "keyword"},
      "document_name": {"type": "keyword"},
      "document_type": {"type": "keyword"},
      "page_start": {"type": "integer"},
      "page_end": {"type": "integer"},
      "page_ids": {"type": "keyword"},
      "text": {"type": "text"},
      "keywords": {"type": "text"},
      "sections": {"type": "keyword"},
      "usc_citations": {"type": "keyword"},
      "acts": {"type": "keyword"},
      "vector": {"type": "dense_vector", "dims": 384, "index": true, "similarity": "cosine"}
    }
  }
}
```

`elastic_indexer.py` does a `helpers.bulk()` load of `chunks_with_embeddings.json` into this mapping (flatten `metadata.entities.acts` → `acts`, `metadata.sections` → `sections`, etc. at index time — don't nest the raw `metadata` blob if it complicates the BM25 field boosting).

`elastic_search.py` builds a single hybrid query using the **native rrf retriever** (Section 2.5), e.g.:

```
{
  "retriever": {
    "rrf": {
      "retrievers": [
        {
          "standard": {
            "query": {
              "multi_match": {
                "query": "<user question>",
                "fields": ["text", "keywords^2", "sections^3", "usc_citations^3", "acts^2"]
              }
            }
          },
          {
            "knn": {
              "field": "vector",
              "query_vector": "<query embedding>",
              "k": 30,
              "num_candidates": 100
            }
          }
        ],
        "rank_window_size": 100,
        "rank_constant": 60
      ]
    },
    "size": 20
  }
}
```

When a graph-derived candidate set exists (Section 8.7.3), add it as a **filter** on `document_id / chunk_id` inside the **standard** sub-retriever's bool query and as a **filter** alongside the **knn** clause, so both sub-retrievers search only the graph-narrowed subset. Boost exact legal-term fields (`sections`, `usc_citations`, `acts`) higher than free-text `text` in the **multi_match**, since exact citation matches should almost always outrank prose overlap. **Fallback:** if the Elastic Cloud trial account cannot use the **retriever** API for any reason, fall back to running a **knn** query and a **bool / multi_match** query as two separate calls and fuse them with the hand-written `rrf.py` (Section 8.7.3) — keep that module either way.

8.6 Query Processing & Entity Detection — `retrieval/query_processor.py`, `retrieval/query_entity_detector.py`

- `query_processor.py`: light touch only — whitespace/quote cleanup, **do not** lowercase or stem (mirrors `cleaner.py`'s philosophy — legal citations in the question must survive intact, e.g. "Section 2302", "26 U.S.C. § 162").

- `query_entity_detector.py` : **import and reuse** `MetadataExtractor`'s `extract_sections`, `extract_usc_citations`, `extract_cfr_citations`, `extract_act_names`, `extract_case_names`, `extract_court_names` (Section 6.4) against the raw question text. Do not reimplement these patterns — importing guarantees a section number extracted from a question and a section number extracted from a chunk are formatted identically after `normalize_entity()`.

8.7 Hybrid Retrieval Orchestration — `retrieval/`

8.7.1 `retrieval/graph_retriever.py`

Takes the entities from 8.6, canonicalizes them, calls the Cypher query from 8.2 (graph expansion, 1-2 hops), returns a candidate `{document_id, chunk_id}` allowlist. If **no** entities were detected in the question (a purely conceptual/semantic question), skip graph filtering entirely and let the hybrid search run unfiltered over the whole index — the graph is a narrowing optimization, not a hard requirement (mirrors the "chapter" note: "graph doesn't replace vector search, it guides it").

8.7.2 `retrieval/hybrid_search.py`

Orchestrates: embed the query (BGE **query** prefix, Section 2.3) → call `elastic_search.hybrid_query()` with the optional graph filter from 8.7.1 → return the top-N hits with their ES score/rank.

8.7.3 `retrieval/rrf.py`

Standalone, dependency-free RRF implementation for the fallback path (8.5) and for merging in future additional ranked lists if the graph is ever turned into a *ranked* signal instead of a filter:

```
def reciprocal_rank_fusion(ranked_lists: list[list[str]], k: int = 60) -> list[tuple[str, float]]:
    scores: dict[str, float] = {}
    for ranked_list in ranked_lists:
        for rank, doc_id in enumerate(ranked_list, start=1):
            scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
    return sorted(scores.items(), key=lambda kv: kv[1], reverse=True)
```

8.7.4 `retrieval/reranker.py`

Load `cross-encoder/ms-marco-MiniLM-L-6-v2` once (`sentence_transformers.CrossEncoder`). Score `(question, chunk_text)` pairs for the top ~20-30 hybrid hits, sort descending, keep the top ~8-10. This is a CPU-friendly step for this corpus size — no batching tricks needed.

8.7.5 `retrieval/context_builder.py`

1. **De-duplicate near-identical overlapping chunks** — necessary precisely because `chunker.py` uses overlapping windows (Section 6.3): if two surviving chunks share, say, >70% token overlap (cheap check: Jaccard similarity on word sets, or compare `page_ids` overlap), keep only the higher cross-encoder-scored one.
2. **Encourage source diversity** — don't let all top-N chunks come from one page/document if a comparable-scoring chunk from a different document type exists; a simple heuristic (cap 2-3 chunks per `document_id` before falling back to next-best from elsewhere) is enough for this project's scale.
3. **Token/word budget** — reserve a fixed budget for system prompt + question + answer (e.g. ~1500 tokens) and fill the remainder with the highest-ranked surviving chunks in order, stopping before the budget is exceeded rather than truncating a chunk mid-sentence. Use a simple `len(text.split()) * 1.3` heuristic for token estimate, or a proper tokenizer if one is already a dependency.
4. **Order**: most relevant chunk first, then supporting evidence, background last — never shuffle randomly.
5. Output: a list of `SOURCE_i` blocks, each `{source_id, document_name, page_start, page_end, chunk_id, text}` — this is the exact structure `prompt_builder.py` (8.8) consumes.

8.8 Prompt Building & LLM Generation — `generation/`

- `prompt_templates.py` : a system prompt that hard-codes the grounding rules ("Answer ONLY using the supplied SOURCE blocks. Never invent citations, section numbers, or case names. If the sources are insufficient, say so explicitly. Always cite the SOURCE IDs your answer relies on.") and a user-prompt template that lists `SOURCE_1..SOURCE_N` (document name + page range + text) followed by the question.
- `llm_client.py` : thin Groq SDK wrapper (`model=GROQ_MODEL` from config, Section 2.4), with retry/backoff on 429s (respect the `Retry-After` header — Groq's free tier is rate limited per-model, per Section 2.2) and a timeout.
- `answer_generator.py` : assembles system + user prompt from `context_builder.py`'s output, calls `llm_client`, returns the raw text.

- `citation_mapper.py` : regex-scan the LLM's answer for `SOURCE_\d+` tokens, map each back to `{document_name, page_start, page_end, chunk_id}` using the same list the prompt was built from (never re-look-up by text similarity — use the exact mapping already in memory from this request). Merge adjacent/contiguous pages from the same document into a range (`16, 17` → `"16–17"` , per the earlier design note on source grouping).
- `safety_checker.py` : reject/flag an answer if it cites a `SOURCE_i` that was never in the prompt, or if the answer is empty, or if the model returned no citations at all for a non-trivial factual question — better to surface "no sufficient evidence" to the evaluation layer than to silently forward an ungrounded answer.
- `answer_formatter.py` : final API-facing shape (Section 5.5 API schema) — `answer` (citation tokens replaced with human-readable `[Document, p. 16–17]` inline or as a separate `sources` list, your call, but be consistent), plus a lightweight **confidence score** — a simple, explainable heuristic is fine here (e.g. normalize and average: cross-encoder top score, fraction of returned SOURCES actually cited by the LLM, and whether graph filtering found any candidates) rather than asking the LLM to self-report confidence.

8.9 API + UI

- `api/fastapi_app.py` : one `POST /ask {"question": "..."}` endpoint returning the Section 5.5 schema; a `GET /health` endpoint that checks Elasticsearch and Neo4j connectivity (useful given AuraDB Free auto-pause, Section 2.2 — the demo should show a clear "waking up the database, retry in a few seconds" state instead of a raw timeout).
- `ui/streamlit_app.py` : a single text box + "Ask" button, showing the answer, then an expandable "Sources" section listing each `SOURCE_i` with document name, page range, and a snippet — this directly satisfies the "Legal Must-Haves" (citations + page-number verification) from the problem statement.

8.10 Evaluation — `evaluation/`

- `golden_set.csv` : load as given (schema, Section 5.5) once it's provided; if it isn't provided yet, **build a small placeholder Golden Set now** (10–15 hand-written Q/A pairs covering all four document types) so the evaluation pipeline can be built and tested before the real Golden Set arrives.
- `retrieval_metrics.py` : implement Precision@K, Recall@K, Hit Rate, MRR, and (optionally, time permitting) nDCG, all from `(retrieved_chunk_ids, expected_document_name)` — a "hit" per question is "at least one retrieved chunk's `document_name` matches `expected_source_document`" unless the Golden Set gives page-level ground truth, in which case tighten the match to `expected_pages` overlap.
- `ragas_evaluator.py` : wire `ragas` up with a **custom LLM wrapper pointed at Groq** (`langchain-groq`'s `ChatGroq` , using `GROQ_MODEL` from config) instead of the OpenAI default, so the judge calls stay on the free tier too. Compute **Faithfulness, Answer Relevancy, Context Precision, Context Recall** per question, using `{question, answer, contexts (the SOURCE texts actually used), ground_truth}` .
- `error_analysis.py` : for every Golden Set question, store `{question, retrieved_docs, expected_doc, answer, ground_truth, retrieval_hit: bool, faithful: bool, error_type}` , where `error_type` is one of: `retrieval_miss` , `hallucination` , `missing_citation` , `wrong_act` , `wrong_judgment` , `graph_miss` , `keyword_miss` , `none` (no error) — this table is the raw material for the evaluation report's qualitative section.
- `report_generator.py` : produce `evaluation_report.md` (or `.pdf` via the same `pdf` skill approach used for this very document) with: an overall metrics table, a short worked example of a correct answer with citations, a short worked example of a failure with its `error_type` , and 2–3 sentences of discussion per metric.

9. Final `config.py` Additions Needed

Add these on top of the existing constants (Section 4.1), grouped by concern:

```

# Graph
GRAPH_DIR = PROCESSED_DATA_DIR / "graph"
RELATIONSHIPS_JSON = GRAPH_DIR / "relationships.json"
OKF_DIR = Path("knowledge")
NEO4J_URI = os.getenv("NEO4J_URI")
NEO4J_USER = os.getenv("NEO4J_USER")
NEO4J_PASSWORD = os.getenv("NEO4J_PASSWORD")

# Embeddings
EMBEDDING_MODEL_NAME = "BAAI/bge-small-en-v1.5"
EMBEDDING_DIM = 384
CHUNKS_WITH_EMBEDDINGS_JSON = PROCESSED_DATA_DIR / "chunks_with_embeddings.json"
BGE_QUERY_PREFIX = "Represent this sentence for searching relevant passages: "

# Elasticsearch
ELASTIC_CLOUD_ID = os.getenv("ELASTIC_CLOUD_ID")
ELASTIC_API_KEY = os.getenv("ELASTIC_API_KEY")
ELASTIC_INDEX_NAME = "legal_rag_chunks"

# Reranking
RERANKER_MODEL_NAME = "cross-encoder/ms-marco-MiniLM-L-6-v2"
RERANK_TOP_K = 10

# LLM
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
GROQ_MODEL = os.getenv("GROQ_MODEL", "openai/gpt-oss-120b") # see Section 2.4 – verify current model before final submission
LLM_CONTEXT_TOKEN_BUDGET = 6500
LLM_RESERVED_TOKEN_BUDGET = 1500

# Evaluation
GOLDEN_SET_CSV = Path("evaluation") / "golden_set.csv"
EVAL_RESULTS_DIR = Path("evaluation") / "results"

```

10. Suggested 3-Day Sprint Plan

Given the tight deadline, this order gets a **thin, working, end-to-end slice** finished as early as possible, then layers in quality — never leave the pipeline in a state where any one stage is "half-built" for long.

Day 1 — Graph + Index (get data searchable). 1. Verify `config.py` and `chunker.py` against Sections 4.1/6.3; add Section 9's config values. 2. Build `relationship_extractor.py` + `graph_schema.py` → `relationships.json` (8.1). 3. Stand up Neo4j AuraDB Free; build `neo4j_client.py` + `neo4j_loader.py`; load the graph (8.2). Sanity-check with 2-3 manual Cypher queries. 4. Build `embedding_generator.py` (8.4) → `chunks_with_embeddings.json`. 5. Stand up Elastic Cloud trial; build `elastic_indexer.py` with the mapping in 8.5; bulk load. 6. **Milestone check:** can you run one `rrf` retriever query by hand in Kibana Dev Tools / curl and get back sensible chunks for a test question? If yes, Day 1 is done.

Day 2 — Retrieval pipeline + generation (get an answer with citations). 1. `query_entity_detector.py` (reuse Section 6.4 regexes) + `graph_retriever.py` (8.7.1). 2. `hybrid_search.py` + `rrf.py` fallback (8.7.2/8.7.3). 3. `reranker.py` + `context_builder.py` (8.7.4/8.7.5). 4. `prompt_templates.py`, `llm_client.py` (using `openai/gpt-oss-120b`, Section 2.4), `answer_generator.py`, `citation_mapper.py`, `safety_checker.py`, `answer_formatter.py` (8.8). 5. **Milestone check:** ask the example question from the problem statement ("How has Section 2302... been interpreted?") end-to-end and get back an answer with correct, verifiable citations. This is the core demo.

Day 3 — API/UI + Evaluation + polish. 1. `fastapi_app.py` + `streamlit_app.py` (8.9) — wrap the Day 2 pipeline, nothing new to design. 2. Build/confirm `golden_set.csv`; `retrieval_metrics.py`; `ragas_evaluator.py` wired to Groq; `error_analysis.py`; `report_generator.py` (8.10). 3. `okf_exporter.py` (8.3) — quick, mechanical, good demo/interview material. 4. Draw the architecture diagram (PDF → Cleaner → Chunker → Metadata → Relationship Extraction → {Elasticsearch, Neo4j} → Hybrid Retrieval → RRF → Rerank → Context Builder → Prompt Builder → Groq LLM → Citation Mapper → Answer, with Evaluation as a parallel branch reading from the same pipeline). 5. Write the evaluation report; record a short demo walkthrough if time allows.

11. Final Target Directory Structure

```
legal-rag-system/
├── config.py
├── .env.example
├── data/raw/{acts,judgments,pov,tax}/*.pdf
├── processed/
│   ├── pages.json
│   ├── clean_pages.json
│   ├── chunks.json
│   ├── chunks_with_metadata.json
│   ├── chunks_with_embeddings.json
│   └── graph/relationships.json
├── ingestion/
│   ├── pdf_parser.py
│   ├── cleaner.py
│   ├── chunker.py
│   └── metadata_extractor.py
├── graph/
│   ├── relationship_extractor.py
│   ├── graph_schema.py
│   ├── neo4j_client.py
│   ├── neo4j_loader.py
│   ├── cypher_queries.py
│   └── okf_exporter.py
├── embeddings/
│   └── embedding_generator.py
├── search/
│   ├── elastic_indexer.py
│   └── elastic_search.py
├── retrieval/
│   ├── query_processor.py
│   ├── query_entity_detector.py
│   ├── graph_retriever.py
│   ├── hybrid_search.py
│   ├── rrf.py
│   ├── reranker.py
│   └── context_builder.py
├── generation/
│   ├── prompt_templates.py
│   ├── llm_client.py
│   ├── answer_generator.py
│   ├── citation_mapper.py
│   ├── safety_checker.py
│   └── answer_formatter.py
├── api/
│   └── fastapi_app.py
├── ui/
│   └── streamlit_app.py
├── knowledge/                                # OKF bundle output
├── evaluation/
│   ├── golden_set.csv
│   ├── retrieval_metrics.py
│   ├── ragas_evaluator.py
│   ├── error_analysis.py
│   ├── report_generator.py
│   └── results/
└── tests/
```

12. Things To Be Careful About (checklist)

- [] Confirm `chunker.py` 's real chunk size/overlap before writing token-budget code (6.3).
- [] Embed queries **with** the BGE instruction prefix, documents **without** it (2.3, 8.4).
- [] Every vector field is 384-dim, everywhere (Elasticsearch mapping, any Neo4j vector index).
- [] `GROQ_MODEL` defaults to `openai/gpt-oss-120b`, not the deprecated `llama-3.3-70b-versatile` — re-check Groq's live model list before final submission (2.4).
- [] Use Elasticsearch's native `rrf` retriever for hybrid search; keep the hand-written `rrf.py` only as a documented fallback (2.5, 8.5, 8.7.3).
- [] Canonicalize every entity string (`normalize_entity`) before it becomes a graph node ID or an Elasticsearch filter term, so `"21 USC 379j-11"` and `"21 U.S.C. 379j-11"` are one node (Section 7).
- [] Use `MERGE`, never `CREATE`, for every Neo4j write (8.2).
- [] Handle Neo4j AuraDB Free's auto-pause with a retry/backoff wrapper (2.2, 8.2).
- [] Respect Groq's free-tier rate limits with backoff on 429s; do not fire one call per chunk in a loop without throttling — this will burn through daily/per-minute quota fast during evaluation runs (2.2, 8.10).

- [] Never let the LLM cite a `SOURCE_i` that wasn't actually in its prompt — validate in `safety_checker.py` (8.8).
- [] De-duplicate overlapping chunks before they reach the LLM, since `chunker.py`'s sliding window guarantees adjacent chunks share text (6.3, 8.7.5).
- [] Keep `.env` out of version control; commit `.env.example` instead (Section 7).
- [] Keep every new pipeline stage in the `Extractor/Generator` + `Pipeline` class shape with the same logging/summary style already established (Section 6.5, 7).

13. Deliverables Checklist (mapped back to the grading milestones)

Deliverable	Where it comes from
Architecture diagram (PDF → ELK/Vector DB/Graph → LLM)	Section 11 tree + Day-3 diagram task
Working demo (UI or API)	Section 8.9 (<code>fastapi_app.py</code> + <code>streamlit_app.py</code>)
Milestone 1 — ingestion preserving page index	Already done (Section 6)
Milestone 2 — vector + keyword + hybrid search, GraphRAG, OKF	Sections 8.1-8.5
Milestone 3 — Q&A, summarization, citations, page references	Sections 8.6-8.9
Milestone 4 — Golden Set run, Retrieval Accuracy + Faithfulness	Section 8.10
Evaluation report	<code>evaluation/report_generator.py</code> output

End of prompt. Give this entire document to the coding agent as-is; it should not need anything else to start implementing Section 8 in the order given in Section 10.